Multimodal Chatbot Web Application - Google Cloud Production Architecture

1. Objective

To build a production-grade, multimodal chatbot web application that can read, analyze, and respond to both text and images in documents, hosted entirely on Google Cloud Platform (GCP). This application is aimed at assisting end users (e.g., customers of a business) with intelligent responses via a conversational interface.

2. Real-World Use Case

Scenario: A customer uploads a scanned insurance form. The chatbot reads both the text and the image content, understands the data, and provides appropriate answers-such as missing fields, next steps, or a summary of the form.

This solution is scalable across sectors like healthcare (interpreting prescriptions), banking (processing KYC documents), and education (grading forms or exam papers).

3. Architecture Overview

Frontend:

- React.js (Latest Version) - Responsive UI with component-based architecture.
- Firebase Hosting - CDN-backed, secure static hosting with built-in SSL and fast global delivery.

Backend:

- Django (Latest Version) - RESTful API server using Django REST Framework.
- Cloud Run (Dockerized) - Fully managed serverless container hosting that scales automatically.

Multimodal AI:

- Vertex AI Gemini 1.5 Pro - Googles foundation model for understanding text and image inputs simultaneously.

Storage & Document Handling:

- Google Cloud Storage (GCS) - Secure file uploads and access.

Database:

- Firestore (NoSQL) - Real-time database for chat sessions and user metadata.
- BigQuery - Scalable analytics and reporting based on chat interactions.

Authentication:

- Firebase Auth - Handles user sign-in via email, phone, Google SSO, etc.

Monitoring and Logging:

- Google Cloud Monitoring & Logging - Centralized monitoring, metrics, and alerting for backend and AI services.

4. Toolchain and Deployment Stack

DevOps Tooling:

- Terraform - Infrastructure as Code (IaC) for provisioning all GCP services.
- GitHub Actions - CI pipelines for linting, testing, and deploying code.
- Cloud Build - CD pipeline to build Docker images and deploy to Cloud Run.

Containerization:

- Docker - Ensures consistent development and deployment environments.

Optional Scalability Layer:

- GKE (Google Kubernetes Engine) - If higher concurrency and job orchestration are needed in the future.

Secrets Management:

- Secret Manager - Secure handling of API keys, DB credentials, and environment variables.

5. Why These Technologies

Google Cloud Focus:

- Using Google Cloud services ensures tight integration, high availability, and scalability.

Cloud Run vs GKE:

- Cloud Run: Best for serverless microservices. Autoscaling, zero management.

- GKE: Recommended only when you need high performance, real-time batch jobs, or microservice orchestration.

Vertex AI Gemini:

- Google's latest multimodal model that handles both image and text inputs natively.

- Eliminates need for separate OCR + NLP pipeline.

Firestore:

- Real-time sync with simple NoSQL document structure - ideal for fast chat session access.

- Supports offline mode and Firebase SDK integration directly into frontend.

BigQuery:

- Scalable analytics and dashboards.

- Easily generate insights from conversation data.

Terraform:

- Ensures repeatable infrastructure provisioning, version control, and auditability.

GitHub Actions + Cloud Build:

- Modern CI/CD pipeline for automatic deployments with rollbacks.

- Ensures developer velocity and stable release pipelines.

Docker:

- Portability across local, staging, and production environments.

- Simplifies dependency management and testing.

Firebase Auth:

- Simple, scalable and secure authentication that integrates directly into Firebase and Google Identity Platform.

6. Production Execution Plan

1. Set up Terraform scripts to provision:
   - Firebase project
   - Firestore database
   - GCS buckets
   - Vertex AI permissions and endpoint
   - Cloud Run services
   - Secret Manager entries

2. Build the React frontend, connect it to Firebase Auth and integrate REST API endpoints.

3. Build and dockerize the Django backend, ensure it supports:

- Secure file upload to GCS
- Session tracking with Firestore
- Integration with Vertex AI Gemini for multimodal input

4. Configure CI/CD:

- Use GitHub Actions to push code changes
- Use Cloud Build to build Docker image and deploy to Cloud Run

5. Monitoring & Logging:

- Enable Cloud Monitoring and Error Reporting on Cloud Run and Vertex AI usage

6. Security Best Practices:

- Store all credentials in Secret Manager
- Enforce HTTPS
- Enable IAM policies and Cloud Audit logs

7. Test in staging environment before promotion to production

7. Final Recommendations

- Begin with Cloud Run for speed, migrate to GKE only when you need fine-grained orchestration.
- Vertex AI is future-proofed and actively evolving - it's the best choice for multimodal use cases.
- Avoid building infrastructure manually - use Terraform from day one.
- Automate testing and deployments early with GitHub Actions and Cloud Build.
- Invest in observability (logs, metrics, alerts) to detect issues proactively.

This architecture ensures a secure, scalable, and modern solution that can handle document understanding and conversational interaction with ease.